

CONTENTS

Glossary of Terms and Abbreviations	ix
1 Introduction	1
1.1 Project Overview	2
1.2 Purpose of this Document	2
1.3 System Context	2
1.4 Document Structure	3
2 Problem and Objectives	5
2.1 Problem Statement	6
2.2 Objectives	6
2.3 Deliverables and Success Criteria	7
3 Requirements and Scope	9
3.1 Actors and Roles	10
3.2 Functional Requirements	10
3.3 Non-Functional Requirements	12
3.4 System Scope	12
3.5 Use Cases	12
3.5.1 Visitor and Registered User Use Cases	12
3.5.2 Administrator Use Cases	13
3.6 Actors in the Implemented Workflow	14
4 System Analysis	17
4.1 Data-Flow Analysis	18
4.1.1 DFD Level 0 – Context Diagram	18
4.1.2 DFD Level 1 – Main Processes	18
4.1.3 DFD Level 2 – Appointment and Reminder Subsystem	19
4.1.4 DFD Level 3 – Slot-Conflict Check	19
4.2 Technology Stack	19
4.3 Analysis of Design Decision Consequences	20
5 System Architecture and Design	23
5.1 Architecture Style	24
5.2 Module Responsibilities	24
5.3 Request Lifecycle	25
5.4 Deployment	26

6	Data Management and Data Model	27
6.1	Database Design	28
6.2	Entities and Collections	28
6.3	Constraints and Indexes	29
6.4	Relationship Coverage Between Models	29
7	Authentication, Authorization, and Security	31
7.1	Content	32
7.1.1	Registration and E-mail Verification	32
7.1.2	Login and Token Issuance	32
7.1.3	Token Verification and Role-Based Access	33
7.1.4	Password Recovery	33
7.2	Security Controls	34
7.3	Security Boundary Verification	35
8	Core Feature Implementation	37
8.1	Digital Pet Identity and QR Code	38
8.2	Adoption Management	38
8.3	Appointments, Notifications, and Reminders	39
8.4	PetGPT – the AI Assistant	40
8.5	Community and Lost-and-Found	41
8.6	Feature Verification Summary	41
9	Implementation	43
9.1	Repository Layout	44
9.2	Backend Implementation	44
9.3	Persistence Pattern	44
9.4	Frontend Structure	45
10	Testing and Validation	47
10.1	Testing Approach	48
10.2	Browser Sweep	48
10.3	Database Persistence Verification	48
10.4	Administrator Tests	49
10.5	Security Tests	49
10.6	Defects Found and Fixed	50
10.7	Test Artifacts	50
11	Development Process	51
11.1	Development Workflow	52
11.2	Phase-Based Roadmap	52
11.3	Timeline and Project Gantt	53
12	Conclusion and Future Scope	55
12.1	Summary of the Delivered System	56
12.2	Observations	56
12.3	Future Scope	56
12.4	Closing Statement	57
	References	59

Appendix A – REST API Summary	63
Appendix B – Configuration	69

LIST OF FIGURES

1.1	FamiPet system context diagram.	3
4.1	DFD Level 0 (context): the FamiPet platform as one process.	18
4.2	DFD Level 1: main processes and data stores of the FamiPet platform. . . .	18
4.3	DFD Level 2: decomposition of the appointment and reminder subsystem. .	19
4.4	DFD Level 3: decomposition of the slot-conflict check.	19
5.1	Layered architecture of the FamiPet backend.	24
5.2	Lifecycle of a protected request through the backend.	25
5.3	Deployment layout of FamiPet.	26
6.1	Entity–relationship diagram of the FamiPet database.	30
7.1	Registration and e-mail verification flow.	32
7.2	Login and JWT issuance flow.	33
7.3	Access-control decision tree (protect and adminOnly).	33
7.4	Password reset flow.	34
8.1	Creation of a pet with automatic QR identity.	38
8.2	Adoption request state diagram.	38
8.3	Adoption application and approval sequence.	39
8.4	Appointment booking, rescheduling, and cancellation flow.	39
8.5	Notification creation and consumption flow.	40
8.6	Synchronization between appointments, notifications, and reminders. . . .	40
8.7	PetGPT request pipeline with fallback.	40
11.1	Project timeline and Gantt chart (July–September 2026).	53

LIST OF TABLES

3.1	User roles and permissions.	10
3.2	Functional requirements.	10
3.4	Visitor and registered-user use cases.	12
3.5	Administrator use cases.	13
3.3	Non-functional requirements.	15
4.1	Technology stack.	19
5.1	Backend route modules and responsibilities.	24
6.1	Mongoose models and collections.	28
7.1	Security controls.	34
10.1	Database persistence evidence.	48
10.2	Security test evidence.	49
11.1	Roadmap phase status.	52
11.2	Key project milestones.	53
12.1	Planned future work.	56

GLOSSARY OF TERMS AND ABBREVIATIONS

Definitions of the technical terms and abbreviations used throughout this black book.

Terms

Term	Definition
API	Application Programming Interface; in FamiPet, the HTTP/REST surface of the backend under <code>/api/</code> .
Bcrypt	A password-hashing function; FamiPet uses cost factor 12 and verifies login passwords against the stored hash.
Bearer token	The JWT sent in the Authorization: Bearer header on protected requests.
Controller	Backend module implementing validation, ownership checks, and business effects for one subsystem.
CORS	Cross-Origin Resource Sharing; the backend origin guard that allows only the configured client, localhost, and private LAN origins.
CRUD	Create, Read, Update, Delete; the basic data operations exposed by the API.
Digital pet identity	The unique <code>petUid</code> plus the QR code generated for each pet.
Document	A record in MongoDB, represented in code by a Mongoose model.
E2E	End-to-end; testing through the real browser and HTTP surface.
JWT	JSON Web Token (RFC 7519); the signed token that authenticates requests.
Mongoose	The object-document mapper used by the backend to define schemas, indexes, and queries on MongoDB.
Multer	Middleware that parses multipart image uploads, restricted to JPEG, PNG, and WebP, up to 5 MB.
Nodemailer	The e-mail library used to send verification and password-reset messages over SMTP.
ObjectId	MongoDB's 24-character hexadecimal document identifier.
ODM	Object-Document Mapper; conversion layer between MongoDB documents and JavaScript objects.
Owner-scoped query	A database query that binds results to the authenticated user (<code>user:req.user._id</code>).
<code>petUid</code>	The unique digital pet identifier embedded in the QR code.
Populate	Mongoose operation that replaces a referenced <code>ObjectId</code> with the referenced document.
<code>protect / adminOnly</code>	The authentication and role middleware pair.
QR code	Quick Response code; encodes the public pet identity payload as a PNG data URL.
REST	Representational State Transfer; the API style used by the platform.
Scheduler	A mechanism that would run periodic jobs; declared in dependencies but not implemented.
SHA-256	The hash used to store e-mail verification and password-reset tokens, so plaintext tokens are never stored.
SMTP	Simple Mail Transfer Protocol, the transport used by Nodemailer.

Abbreviations

Abbreviation	Meaning
HTTP(S)	HyperText Transfer Protocol (Secure)
JSON	JavaScript Object Notation
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
URI	Uniform Resource Identifier
DB	Database
LAN	Local Area Network
AI	Artificial Intelligence
QA	Quality Assurance
UI	User Interface
UX	User Experience

CHAPTER 1

INTRODUCTION

This chapter introduces FamiPet, a pet-care and adoption platform. It describes the purpose of the project, the scope of this black book, the overall system context, and the document structure.

1.1 Project Overview

FamiPet is a full-stack web application that supports pet owners, prospective adopters, and platform administrators. The application lets registered users maintain profiles for their pets, generate a digital pet identity with a QR code, record health and vaccination information, book veterinary appointments, set medication and care reminders, participate in a community, report lost or found animals, and apply to adopt pets listed on the platform. An administrator can moderate content, manage users, and process adoption requests.

The system is built as a client–server application. The backend is a Node.js [1] application written with Express [2] that exposes a REST API and persists all data in MongoDB through the Mongoose object-document mapper. The frontend is a React and TypeScript web application built with Vite and Tailwind CSS; it communicates with the backend over HTTP, and the backend remains the source of truth for all data and business rules.

1.2 Purpose of this Document

This black book is a formal academic record of the FamiPet project. It documents, from the actual implementation:

- the problem the project addresses and its objectives;
- the functional and non-functional requirements and the system scope;
- the analysis and design of the system;
- the system architecture and deployment;
- the data model stored in MongoDB;
- the authentication, authorization, and security mechanism;
- the core features and their workflows;
- the implementation structure and technology stack;
- the testing performed and its evidence;
- the development process followed;
- conclusions and future work.

Every statement about behaviour, data, security, and testing in this document is derived from the verified implementation in the repository. Planned work, such as the remaining roadmap phases described in Chapter 11.3, is explicitly identified as planned rather than as delivered functionality.

1.3 System Context

Figure 1.1 shows the context of the system. The backend API is the central component: it receives HTTP requests from the frontend, applies authentication and ownership rules, and persists and retrieves data in MongoDB. It also integrates with external services for e-mail delivery, AI-powered pet care assistance, and image processing.

The context diagram shows four external parties. The *React web client* interacts with the platform through the public HTTP API; the *MongoDB* database stores all records; the *e-mail service* (Nodemailer over SMTP) delivers account verification and password-reset messages; and the *Google Gemini* AI service provides the conversational pet-care assistant. Uploaded images are stored on Cloudinary when credentials are configured and served locally from the `uploads` directory otherwise.

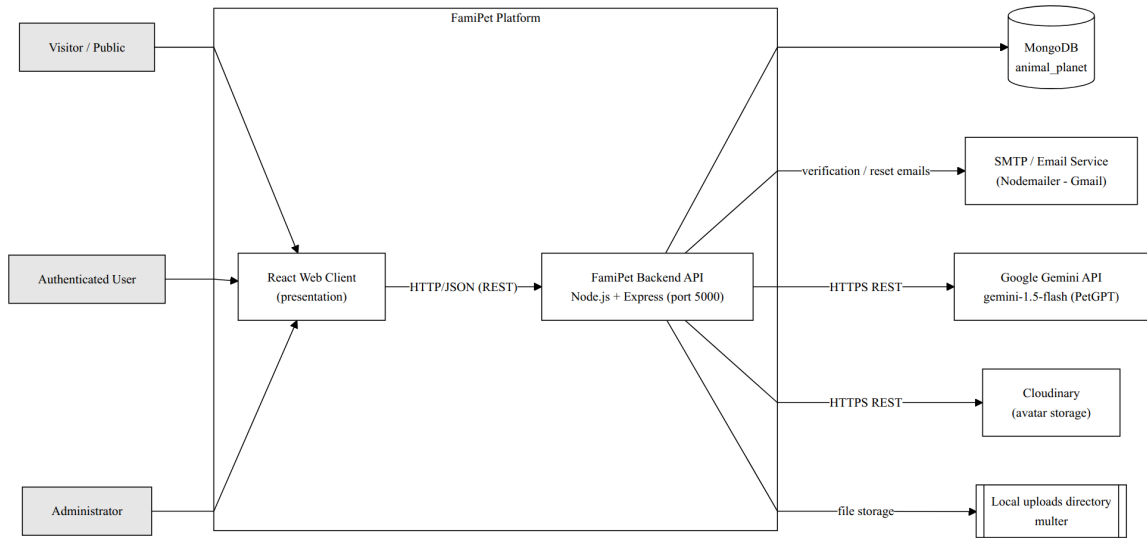


Figure 1.1. FamiPet system context diagram.

1.4 Document Structure

The black book is organised as follows. Chapter 2 defines the problem and objectives. Chapter 3 presents the requirement specification and scope, including functional and non-functional requirements, user roles, and use cases. Chapter 4 analyses the system and the technology stack. Chapter 5 describes the architecture and design. Chapter 6 documents the MongoDB data model. Chapter 7 describes authentication, authorization, and security. Chapter 8 explains the core features and their workflows. Chapter 9 describes the implementation structure. Chapter 10 records the testing and validation evidence. Chapter 11 documents the development process. Chapter 12 presents the conclusion and future scope. The list of terms and abbreviations appears before the main matter; references and two appendices conclude the black book.

CHAPTER 2

PROBLEM AND OBJECTIVES

This chapter defines the problem that FamiPet addresses, the objectives of the project, and the deliverables that define a successful implementation.

2.1 Problem Statement

Pet ownership involves several practical responsibilities that are today spread across separate tools and sources: remembering vaccination schedules, tracking medical history, keeping appointments with veterinarians, organising daily care like feeding and medicine, proving a pet's identity when it is lost, and managing the process of adopting an animal responsibly. Owners keep this information informally or in unrelated documents, so records are lost, reminders are missed, and a pet's health history is unavailable at the moment it is needed. People looking for a pet have similar difficulties: there is no single place to see animals available for adoption and to apply for them, or to report and search lost or found animals.

FamiPet addresses this by providing one integrated platform in which:

- a user registers, verifies the account, and manages a personal profile;
- a user records pets, generating a unique digital pet identity and a QR code that encodes public pet information;
- health records and vaccinations are kept per pet, with ownership enforced by the backend;
- veterinary appointments can be booked against a veterinarian's availability, with automatic notifications and reminders;
- reminders for feeding, medicine, grooming, and care can be created and completed;
- users can post to a community, like and comment on posts, and report lost or found animals;
- users can apply to adopt pets, and administrators can approve or reject those applications, automatically changing the pet's status; and
- an AI assistant answers pet-care questions and gives advice based on the user's actual pet records.

The problem is therefore the need for an authenticated, backend-driven pet-care system in which all user data is owned by the correct user, is persisted in a real database, and is protected by server-side access control.

2.2 Objectives

The objectives of the FamiPet project are:

1. To build a full-stack web application for pet care and adoption with an authenticating backend and a database-backed data store.
2. To provide a secure authentication mechanism based on e-mail verification, password hashing, and signed bearer tokens.
3. To enforce ownership and role-based authorization on the backend so that users can only access their own resources and only administrators can perform administrative actions.
4. To implement a digital pet identity with a unique pet identifier and a QR code that can be read to obtain public pet information.
5. To implement pet health management, including health records and vaccinations, tied to the authenticated owner's pets.
6. To implement appointment booking with veterinarian slot conflict detection and automatic notification and reminder generation.
7. To implement a reminder system with per-owner, per-pet reminders that persist and can be completed.

8. To implement adoption management with pending, approved, and rejected states and automated pet status transitions and notifications.
9. To implement community and lost-and-found features with moderation by administrators.
10. To integrate an AI pet-care assistant that uses the owner's real pet data and falls back to safe, generic guidance when the AI service is unavailable.
11. To validate the system through real end-to-end tests, database persistence checks, and security boundary tests.

2.3 Deliverables and Success Criteria

The project deliverables are:

- the backend API (Express, MongoDB, Mongoose) at `backend/`;
- the React frontend application (`frontend-react/`) with TypeScript, Vite, and Tailwind CSS;
- project documentation, including this black book, the development roadmap (`ROADMAP.md`), and the maintenance rules in `AGENTS.md`;
- a seeded database containing default breeds, an administrator account, and a demo user for evaluation.

A phase is considered successful when the functionality is implemented, connects to the persisted database, passes the associated tests, and is committed and pushed to the repository. Feature phases are tracked in a checklist: at the time of writing, the backend is feature-complete and the React frontend implements roadmap phases 1–22, while phases 23–29 remain planned (Chapter 10.7).

CHAPTER 3

REQUIREMENTS AND SCOPE

This chapter specifies the functional and non-functional requirements of FamiPet, the user roles and their permissions, the scope of the system, and the use cases that the implementation supports.

3.1 Actors and Roles

The system distinguishes three actors, defined by the actual implementation.

Role	Stored as	Permissions enforced by the backend
Visitor	no account	Browse public content: pets, breeds, veterinarian directory, community posts, and lost-and-found reports. Cannot create or modify any resource.
Registered user	<code>User.role = "user"</code>	All visitor permissions plus account management, pet management with QR identity, health records, vaccinations, appointments, reminders, favorites, community posts, lost-and-found reports, adoption applications, notifications, and the AI assistant.
Administrator	<code>User.role = "admin"</code>	All user permissions plus the admin panel: dashboard statistics, user management and blocking, pet moderation, adoption approval/rejection, community and lost-and-found moderation, and breed and veterinarian catalogue administration.

Table 3.1. User roles and permissions.

The role is stored in the `User` document and is never accepted from the client. The `protect` middleware authenticates the request from the bearer token, and the `adminOnly` middleware verifies `req.user.role === "admin"` before any administrative route executes (Section 7.2).

3.2 Functional Requirements

The functional requirements below are grouped by subsystem. Each requirement is directly supported by a verified controller and route in the backend.

Table 3.2. Functional requirements.

ID	Subsystem	Requirement
FR-01	Authentication	The system shall allow a user to register with name, e-mail, and password; store the password as a bcrypt hash; and require e-mail verification before the account can log in.
FR-02	Authentication	The system shall issue a signed JWT on successful login and require it as a bearer token on protected routes.
FR-03	Authentication	The system shall support password reset through a time-limited, single-use token sent by e-mail, and password change with the current password.
FR-04	Profile	The user shall be able to read and update own profile (name, phone, address, city, avatar).
FR-05	Pets	The user shall be able to create, list, view, update, and delete own pets, and only own pets for protected operations.

FR-06	Pets	On creation, the system shall generate a unique digital pet identifier and a QR code encoding public pet information.
FR-07	Pets	Public listing shall support filtering by species, breed, gender, and status, full-text search, and sorting.
FR-08	Health	The user shall be able to create, read, update, and delete health records for own pets.
FR-09	Vaccinations	The user shall be able to create, read, update, delete, and list upcoming vaccinations for own pets.
FR-10	Appointments	The user shall be able to book, list, update, and cancel appointments for own pets with an active veterinarian.
FR-11	Appointments	The system shall reject a double booking when the same veterinarian already has a pending or confirmed appointment at the same date and time.
FR-12	Appointments	Booking an appointment shall automatically create a notification and a non-duplicate reminder for the owner.
FR-13	Reminders	The user shall be able to create, list, update, complete, and delete reminders; reminders are scoped to the owner.
FR-14	Notifications	The user shall be able to list, mark as read, mark all as read, and delete own notifications, and read the unread count.
FR-15	Adoption	A user shall be able to apply to adopt an available pet with contact and motivation details, and a duplicate pending request for the same pet shall be rejected.
FR-16	Adoption	An administrator shall be able to approve or reject an adoption request; approval automatically marks the pet as adopted and sets its status to adopted .
FR-17	Adoption	Any status change of an adoption request shall create a notification for the applicant.
FR-18	Favorites	The user shall be able to add and remove favorite pets; the same user and pet pair must not be duplicated.
FR-19	Community	Any visitor shall be able to read active posts; a signed-in user shall be able to create, update, and delete own posts, and like and comment on posts.
FR-20	Lost and Found	A signed-in user shall be able to create, update, and delete lost/found reports, and visitors shall be able to view active reports.
FR-21	AI assistant	A signed-in user shall be able to ask the PetGPT assistant questions; the answer shall consider the user's pets and fall back to safe generic guidance when the AI service is unavailable.
FR-22	AI advice	A signed-in user shall be able to request rule-based advice for an own pet based on its vaccination status, age, and weight.
FR-23	Admin	The administrator shall be able to view dashboard statistics, list and block users, delete pets, moderate community and lost-and-found content, approve or reject adoptions, and manage breeds and veterinarians.

FR-24	E-mail	The system shall send verification, resend-verification, and password-reset e-mails through Nodemailer, and construct links using the origin of the request.
-------	--------	--

The functional requirements table above is enforced through the route handlers and controllers described in Chapters 5–9.

3.3 Non-Functional Requirements

3.4 System Scope

The implemented scope of the system is the authenticated pet-care and adoption workflow described by the functional requirements above, with administration capabilities. Outside the current scope are features mentioned in configuration or documentation but not present in the implementation, including:

- the OpenAI adapter advertised in `backend/.env.example` (`PETGPT_PROVIDER=openai`); the implemented assistant uses only the Google Gemini provider;
- background job processing: the `node-cron` and `socket.io` packages are declared dependencies but are not used by any application code, so no scheduled jobs, queues, or workers exist;
- real-time push notifications (notifications are database-bound and read through the API);
- the remaining roadmap phases 23–29 (API integration layer, authentication consolidation, UI completion, visual and functional regression, and Docker/Nginx integration, followed by client consolidation).

3.5 Use Cases

The use cases consolidate the functional requirements. The catalogue is organised by actor.

3.5.1 Visitor and Registered User Use Cases

Table 3.4. Visitor and registered-user use cases.

Use case	Requirement	Description and key flow
UC-01 Reg- ister	FR-01	Submit name, e-mail, password; backend normalizes the e-mail, validates format, hashes the password, stores a hashed verification token, and e-mails a verification link.
UC-02 Ver- ify e-mail	FR-01	Following a clicked link, the token is hashed and matched; the account becomes verified and the token is cleared.
UC-03 Lo- gin	FR-02, FR-03	Credentials are compared against the stored hash; blocked and unverified accounts are rejected; a JWT is returned.

UC-04 Reset password	FR-03	A link with a single-use token is e-mailed; the new password is hashed and the token cleared.
UC-05 Manage profile	FR-04	Read or update own profile fields.
UC-06 Manage pets	FR-05, FR-06, FR-07	Create pets (QR identity generated), list own pets, view public pets, update or delete only own pets.
UC-07 Manage health	FR-08, FR-09	Create and manage health records and vaccinations for own pets.
UC-08 Book appointments	FR-10–FR-12	Book with slot conflict check; automatic notification and reminder; reschedule or cancel keeps the reminder in sync.
UC-09 Manage reminders	FR-13	Create, complete, and manage own care reminders.
UC-10 Manage notifications	FR-14	View, mark read, and manage own notifications.
UC-11 Apply for adoption	FR-15–FR-17	Submit an adoption application; track requests; receive notifications on approval or rejection.
UC-12 Favourites	FR-18	Toggle pets in a personal favourites list.
UC-13 Community	FR-19	Read, create, update, delete posts; like and comment.
UC-14 Lost and found	FR-20	Create, update, resolve, and delete own reports; view active reports.
UC-15 AI assistant	FR-21, FR-22	Ask the pet-care assistant; request advice for an own pet.

3.5.2 Administrator Use Cases

Table 3.5. Administrator use cases.

Use case	Requirement	Description
UC-16 Dashboard	FR-23	Aggregate user, pet, adoption, lost-and-found, and community statistics.
UC-17 User management	FR-23	List users, view recent users, block/ unblock, and delete users.
UC-18 Pet moderation	FR-23	List all pets with owner information; delete pet records.

UC-19 Adoption moderation	FR-16, FR-17	Approve or reject adoption applications; approval flips the pet to adopted .
UC-20 Content mod- eration	FR-23	Activate/deactivate and delete community posts; re- solve and delete lost-and-found reports.
UC-21 Catalogue management	FR-23	Create, update, and delete breeds and veterinarians.

3.6 Actors in the Implemented Workflow

The system context and actor interactions are summarized in the context diagram of Chapter 1. Community, lost-and-found, and veterinarian reading endpoints are public, which matches the two-level access model: authenticated creation and ownership-scoped management for content, and public read access to active community, lost-and-found, and veterinary records.

Category	Requirement
Security	Passwords stored as bcrypt hashes (cost factor 12); tokens (e-mail verification, password reset) are single-use, time-limited, and stored only in SHA-256 hashed form.
Access control	All protected operations are scoped to the authenticated user; administrative operations are additionally gated by role.
Reliability	The system must not rely on the availability of external services: AI and image-hosting failures degrade gracefully to stored, safe alternatives.
Validation	The backend validates required fields, e-mail format, password length, resource ownership, and invalid IDs and returns appropriate HTTP errors (400, 401, 403, 404, 500).
Performance	JSON responses are compressed with compression ; appointment double-booking is prevented by a database-backed conflict query with compound indexing on the veterinarian, date, and time.
Usability	The frontend is responsive and supports both light and dark theme variants; migrated React pages are verified at 390, 768, and 1440 pixels.
Maintainability	The backend follows a layered structure (routes, controllers, models, middleware, config) and a phase-based development roadmap with verification before each commit.

Table 3.3. Non-functional requirements.

CHAPTER 4

SYSTEM ANALYSIS

This chapter analyses the FamiPet system: the data flow between client, API, and database, and the technology stack with the responsibility of each component.

4.1 Data-Flow Analysis

This section presents the data-flow diagrams (DFDs) of the system following the structured-analysis convention [3, 4]: each level decomposes a process of the previous level, and the notation distinguishes external entities, processes, and data stores.

The bounded system is the FamiPet platform. Every data-producing process in the application follows the same verified path: the client sends an HTTP request to a route, middleware authenticates the caller, a controller validates the request and applies ownership rules, the Mongoose model persists or reads a document in MongoDB, and the API returns a JSON response to the client.

4.1.1 DFD Level 0 – Context Diagram

Figure 4.1 shows the whole system as a single process (process 0) connected to the three external entities with which it exchanges data: the end user, the e-mail service, and the AI service.

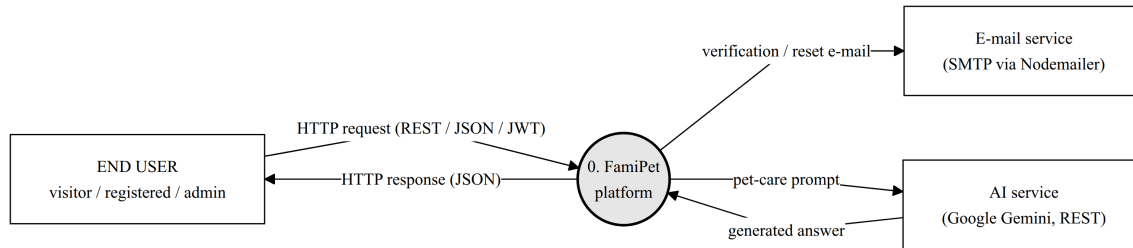


Figure 4.1. DFD Level 0 (context): the FamiPet platform as one process.

4.1.2 DFD Level 1 – Main Processes

Figure 4.2 decomposes process 0 into the process groups exposed by the route modules and shows the single store, MongoDB, with which every process interacts. Two external data sinks are visible: the e-mail service, written to only by the authentication process, and the AI service, which is reached by the AI process (process 11) and the Google Gemini endpoint.



Figure 4.2. DFD Level 1: main processes and data stores of the FamiPet platform.

In practice each subsystem respects the same flow, e.g. an appointment booking produces a request on `POST /api/appointments`, and the controller writes an `Appointment` document, synchronously creates a `Notification` document and a `Reminder` document, and then responds with the created record. This flow is the reason the application requires no background workers: all derived records (notifications, reminders) are created at request time, in the same transaction context provided by the database connection.

4.1.3 DFD Level 2 – Appointment and Reminder Subsystem

Figure 4.3 decomposes the appointment process (process 4) of Figure 4.2 into the concrete controller steps: field and identifier validation, ownership and active-veterinarian checks, the slot-conflict check, creation of the **Appointment**, and the synchronous creation of the **Notification** and **Reminder** records with de-duplication of the reminder.

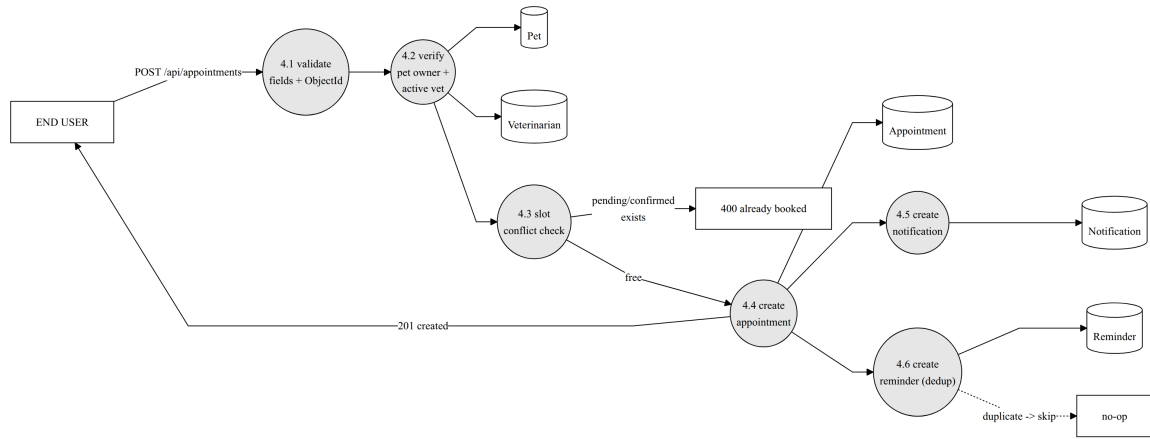


Figure 4.3. DFD Level 2: decomposition of the appointment and reminder subsystem.

4.1.4 DFD Level 3 – Slot-Conflict Check

Figure 4.4 further decomposes the conflict check (process 4.3) that enforces the double-booking rule of FR-11: the query is built against the veterinarian, date, and time; if any existing appointment carries the status **pending** or **confirmed**, the request is rejected with a 400 response, otherwise flow proceeds to creation.

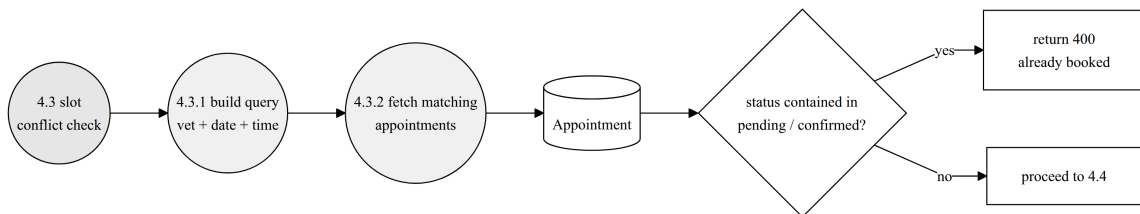


Figure 4.4. DFD Level 3: decomposition of the slot-conflict check.

4.2 Technology Stack

The table below lists the technologies verified in the project implementation and how each is used.

Table 4.1. Technology stack.

Technology	Type	How it is used
Node.js	Runtime	Executes the backend (<code>server.js</code>) on a single process, port 5000 by default.

Express 4	Web framework	Routing, JSON bodies (50 MB limit), static files, URL-encoded bodies, final error and 404 handlers.
MongoDB	Database	Persistent document store (<code>animal_planet</code> by default).
Mongoose 8	ODM	13 models mapped to collections; schema validation, enum constraints, unique compound indexes, pre-save hashing hooks, query methods.
jsonwebtoken	Auth	Signs and verifies the bearer JWT with a 30-day expiry by default.
bcryptjs	Auth	Hashes passwords with cost factor 12 and compares them on login.
crypto	Auth	Generates 32-byte random tokens, hashed with SHA-256 before storage, for e-mail verification and password reset.
nodemailer	E-mail	Sends HTML e-mails through SMTP (Gmail service by default).
multer	Upload	Accepts image uploads (JPEG/PNG/WebP, max 5 MB) to the <code>uploads/</code> directory.
qrcode	QR	Generates a PNG data-URL QR code for the digital pet identity.
cloudinary	Images	Optional image-hosting integration read from environment configuration.
@google/generative-ai	AI	Defines the Gemini model, but the implementation calls the REST endpoint directly with <code>fetch</code> .
morgan	HTTP logging	Development request logging.
compression	HTTP	GZIP-style response compression.
helmet	Security	Sets security HTTP headers, with cross-origin-resource policy relaxed so uploaded images render on the frontend origin (Section 7.2).
cors	Security	Origin-guard that permits the configured client origin, localhost, and private LAN ranges.
node-cron	Declared	Declared but unused; no scheduled jobs exist.
socket.io	Declared	Declared but unused; no real-time channel exists.
React + TypeScript	Frontend	The active React [5] client stack of <code>frontend - react</code> .
Vite + Tailwind CSS	Frontend tooling	Build tooling and styling for the React client.

4.3 Analysis of Design Decision Consequences

Two configuration observations are worth recording because they affect how the system behaves at runtime.

First, the PetGPT assistant is implemented against Google’s Gemini REST endpoint (system instruction, contents, 25-second timeout, and a keyword fallback). The OpenAI

PETGPT_OPENAI_* variables in `backend/.env.example` are not implemented, so the active provider is always Google when a key is present. Second, reminders are not pushed asynchronously: the `Reminder` collection records date and time, but no background scheduler delivers notifications; the reminders are presented to the user through the reminders API. Both facts are documented here so the reader does not mistake declared dependencies for delivered behaviour.

CHAPTER 5

SYSTEM ARCHITECTURE AND DESIGN

This chapter describes the logical architecture of FamiPet, the request lifecycle, the deployment layout, and the responsibility of each component.

5.1 Architecture Style

FamiPet is a client–server application whose backend is a *modular monolith*: one Express process exposes all sixteen route modules, and all business logic, validation, and persistence live in that process against a single MongoDB store. There are no microservices, no message broker, and no background workers. This architecture was chosen because the functional requirements are interdependent (appointments create reminders and notifications; adoption approval updates the pet and notifies the applicant) and the data is shared, so a single transaction context keeps the flows simple, consistent, and testable.

Figure 5.1 shows the layered structure of the backend as it is implemented. The presentation boundary is the HTTP API consumed by the web client. Gateway middleware sets the security and parsing context. The routing layer maps the sixteen route modules, and access-control middleware (`protect`, `adminOnly`, `upload`) enforces identity, roles, and file constraints. The controller layer implements the business logic. The data layer maps the 13 Mongoose models to MongoDB collections. Two external boundaries are drawn: e-mail and AI.

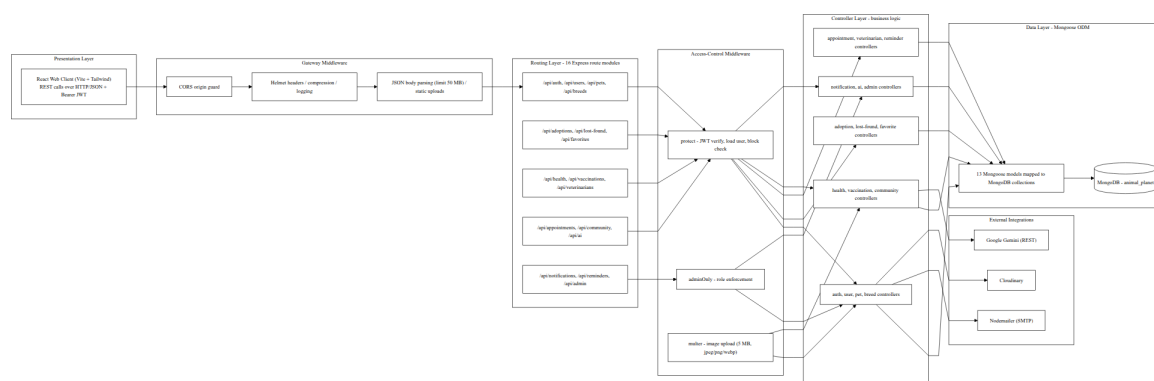


Figure 5.1. Layered architecture of the FamiPet backend.

The layered diagram reflects the actual module boundaries of the repository. Cross-cutting concerns (CORS, helmet, compression, JSON parsing) run before any route module. The `protect` middleware runs on every protected route, the `adminOnly` middleware on administrative routes, and the Multer [6] upload middleware only on routes that accept images.

5.2 Module Responsibilities

The table below lists the sixteen Express route modules and the central responsibilities of their controllers.

Table 5.1. Backend route modules and responsibilities.

Route module	Responsibility
<code>/api/auth</code>	Register, verify and resend e-mail verification, login, forgot/reset password, current user, profile update, password change.
<code>/api/users</code>	Public user lookup, favorites toggle, avatar upload.
<code>/api/pets</code>	Public pet listing, featured pets, own pets, pet CRUD with owner enforcement, QR generation.

<code>/api/breeds</code>	Public breed catalogue; admin create/update/delete.
<code>/api/adoptions</code>	Adoption applications, own applications, admin listing and status updates with pet transition and notification.
<code>/api/lost-found</code>	Public reports, owner CRUD, admin status updates and deletion.
<code>/api/health</code>	Owner-scoped health records per pet.
<code>/api/vaccinations</code>	Owner-scoped vaccination records and upcoming list.
<code>/api/favorites</code>	Owner-scoped favourite pets (unique user–pet pairs).
<code>/api/veterinarians</code>	Public directory; admin CRUD.
<code>/api/appointments</code>	Booking with slot conflict check, auto-notification and auto-reminder, reschedule and cancel with reminder sync.
<code>/api/community</code>	Public posts, owner CRUD, likes, comments; admin moderation.
<code>/api/ai</code>	<code>/ask</code> (PetGPT over Gemini with fallback) and <code>/advice</code> (rule-based per-pet advice).
<code>/api/notifications</code>	Owner-scoped notification read, unread, mark read/all-read, delete.
<code>/api/reminders</code>	Owner-scoped reminders CRUD and completion.
<code>/api/admin</code>	Dashboard statistics, user/pet/lost-found/community administration.

5.3 Request Lifecycle

Figure 5.2 shows the lifecycle of a protected request, using the example of an authenticated controller that owns a resource. The sequence is identical for every protected subsystem; the ownership predicate is the only part that varies (see Chapter 6.4).

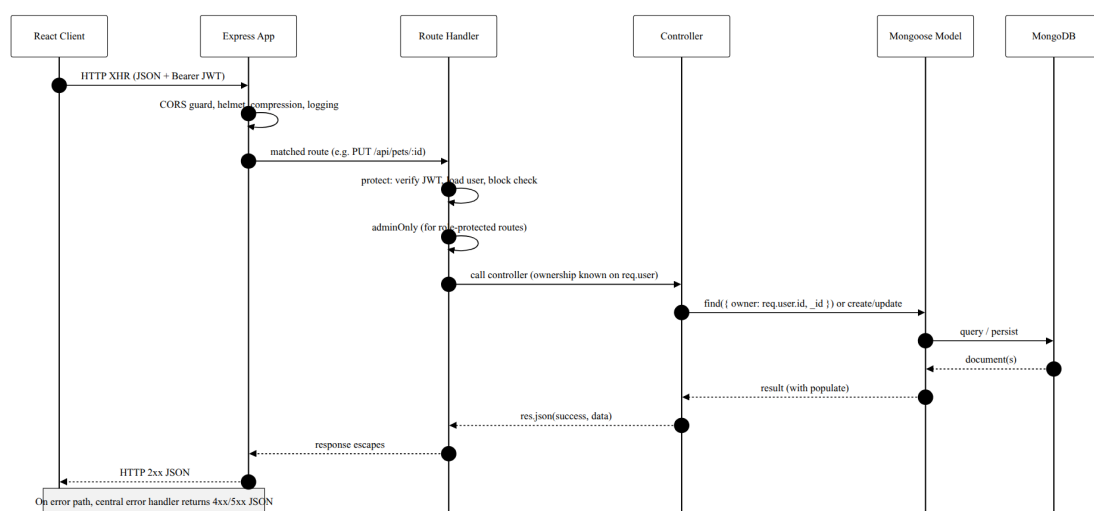


Figure 5.2. Lifecycle of a protected request through the backend.

Once the token is verified and the user is loaded, owner-scoped queries bind the resource to `req.user._id`: for example, appointments are queried with `{user: req.user._id}`, health records and vaccinations the same way, and pet operations compare `pet.owner` to

`req.user.id`. This binding is what makes user isolation enforceable without trusting any client-supplied identifier.

5.4 Deployment

Figure 5.3 shows the deployment layout used during the project. The backend process and the frontend are served on different ports; the database runs as a separate service, optionally with local tooling (Compass) for inspection. The backend also serves the `uploads` directory statically so uploaded images resolve from the API. Credentials for the optional external image-hosting platform Cloudinary [7] are declared in the configuration but the current implementation stores uploads locally on disk.

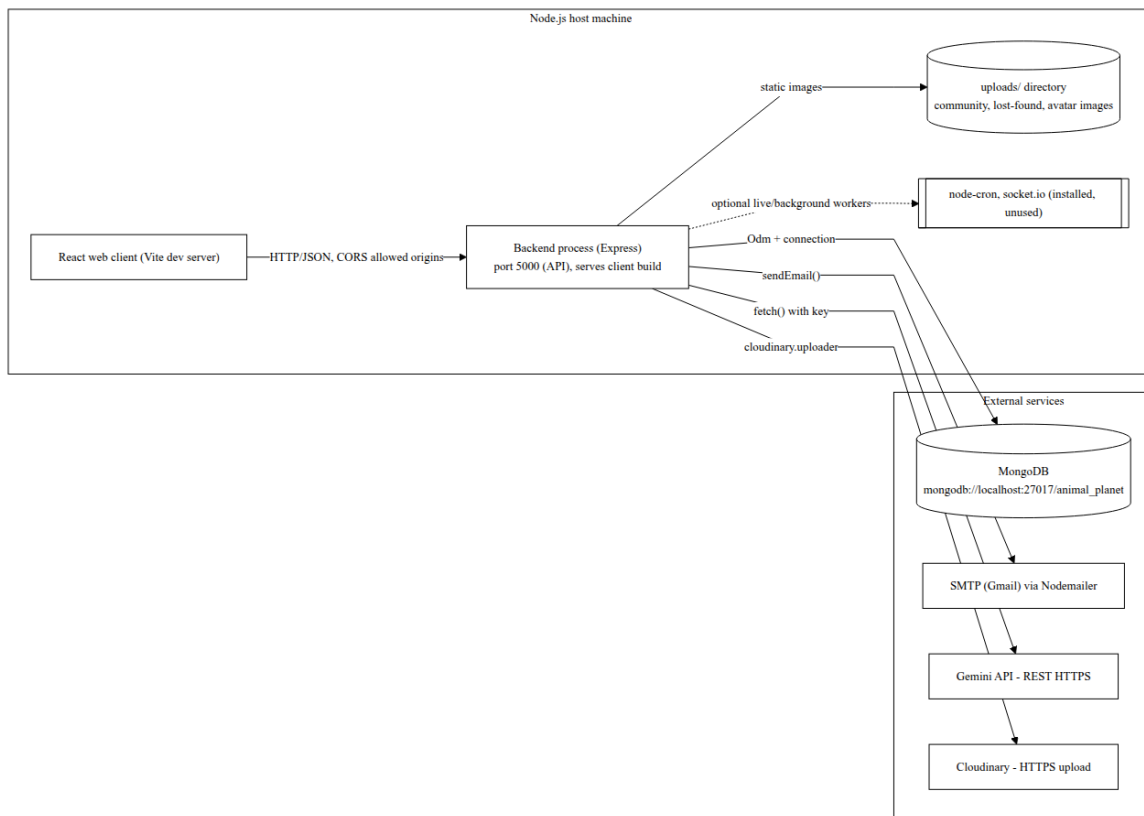


Figure 5.3. Deployment layout of FamiPet.

The server binds to the environment-configured port (5000 by default). The client origin is taken from `CLIENT_URL` (5000-series Live Server/Vite by default) and is also used to fall back to a locally served page for the `verify-email` and `reset-password` routes. MongoDB connects through `MONGODB_URI`, defaulting to `mongodb://localhost:27017/animal_planet`.

CHAPTER 6

DATA MANAGEMENT AND DATA MODEL

This chapter documents the MongoDB data model: the thirteen entities, their relationships, and the constraints and indexes that protect data integrity.

6.1 Database Design

FamiPet uses MongoDB [8] as its primary store. The schema is defined in thirteen Mongoose [9] models, all located in `backend/models/`. Figure 6.1 shows the entity–relationship view of the store. Dashed lines mark references whose target documents are populated through Mongoose `populate`.

The central entity is **Pet**. Every pet belongs to exactly one **User** (the **owner**) and references exactly one **Breed**. Users may star pets through **Favorite**, whose compound unique index guarantees that the same user cannot favourite the same pet twice. Health records and vaccinations belong to a user and a pet, appointments to a user, a pet, and a veterinarian, and reminders to a user with an optional pet reference. Adoption requests belong to a user and a pet, and their status is managed by administrators. Community posts embed arrays of likes and comments; lost-and-found reports record animal and contact information. Notifications are per-user documents that record read state.

6.2 Entities and Collections

Table 6.1. Mongoose models and collections.

Model	Collection	Key fields
User	<code>users</code>	name, email (unique), password (select false), avatar, phone, address, city, role (user/admin), isVerified, isBlocked, emailVerificationToken + expire (SHA-256), resetPasswordToken + expire (SHA-256), pets[], favorites[], adoptionRequests[], notifications[].
Pet	<code>pets</code>	owner, breed, name, species (dog/cat/bird/rabbit/fish/ other), gender, age, weight, color, vaccinated, adopted, status (available/adopted/lost/inactive), images[], description, views, qrCode, petUid.
Breed	<code>breeds</code>	name (unique), species, origin, lifespan, weight/height range, temperament[], exerciseRequirements, groomingGuide, commonDiseases[], suitableEnvironment, description, images[], popularity, isActive.
Adoption	<code>adoptions</code>	pet, user, fullName, phone, address, occupation, experienceWithPets, reasonForAdoption, status (Pending/Approved/Rejected).
Appointment	<code>appointments</code>	user, pet, veterinarian, date, time, type (check up/vaccination/surgery/emergency/grooming/consultation), status (pending/confirmed/completed/cancelled/no-show), symptoms, notes, prescription, diagnosis, fee, isPaid.
Notification	<code>notifications</code>	user, title, message, type (adoption/appointment/vaccination/health/system/other), isRead.

Reminder	reminders	user, pet, title, type (feeding/medicine/ vaccination/grooming/appointment/exercise/custom), description, date, time, frequency (once/daily/weekly/monthly), isActive, isCompleted.
HealthRecord	healthrecords	user, pet, diagnosis, treatment, doctor, hospital, prescription, visitDate, nextVisit, notes.
Vaccination	vaccinations	user, pet, vaccineName, doseNumber, vaccinationDate, nextDueDate, veterinarian, hospital, notes, status (Pending/Completed).
Favorite	favorites	user, pet, with a unique compound index.
Veterinarian	veterinarians	name, email, phone, specialization[], qualifications[], experience, clinic, address, city, image, rating, reviews[], availability[], isActive, consultationFee.
CommunityPost	communityposts	user, title, content, category, image, likes[], comments[] (embedded), isActive.
LostFound	lostfounds	user, type (lost/found), petName, species, breed, gender, color, description, location, date, contactName, contactPhone, images[], status (active/resolved).

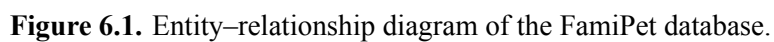
6.3 Constraints and Indexes

The schema enforces integrity at three levels.

- **Required and enum constraints.** Required fields are declared in each schema, and enumerated fields (`role`, `species`, `status`, message and reminder types, appointment type and status, adoption status) are constrained to the documented values. Violations surface as Mongoose `ValidationErrors` and are returned as HTTP 400.
- **Unique indexes.** `User.email` is unique; `Breed.name` is unique; `Favorite` carries a compound unique index on `{user, pet}` that prevents duplicate favourites. Duplicate key violations (error code 11000) are mapped to HTTP 400 by the error handler.
- **Pre-save hooks.** The `User` schema hooks `save` and hashes the password with `bcrypt` (cost factor 12) whenever the password field has been modified. The `password` field uses `select: false` so it is excluded from all queries unless the controller explicitly selects it (login, change-password, reset flows).

6.4 Relationship Coverage Between Models

The ER diagram and the entity table together demonstrate that every user-specific document carries the owning user reference and is queried with that reference. This pattern is the data-level backbone of the access control described in Chapter 6.4: a record created by user A is stored with `user: A`, so every later owner-scoped query naturally excludes user B's data even before any authorization logic runs.



CHAPTER 7

AUTHENTICATION, AUTHORIZATION, AND SECURITY

This chapter explains how FamiPet establishes user identity, how it enforces authorization, and the security controls applied at the HTTP and storage layers.

7.1 Content

Authentication is handled entirely by the backend. The flow has three stages: account creation with e-mail verification, login with a signed token, and token-based access control for every protected route. Figures 7.1 through 7.4 show the four authentication workflows as implemented.

7.1.1 Registration and E-mail Verification

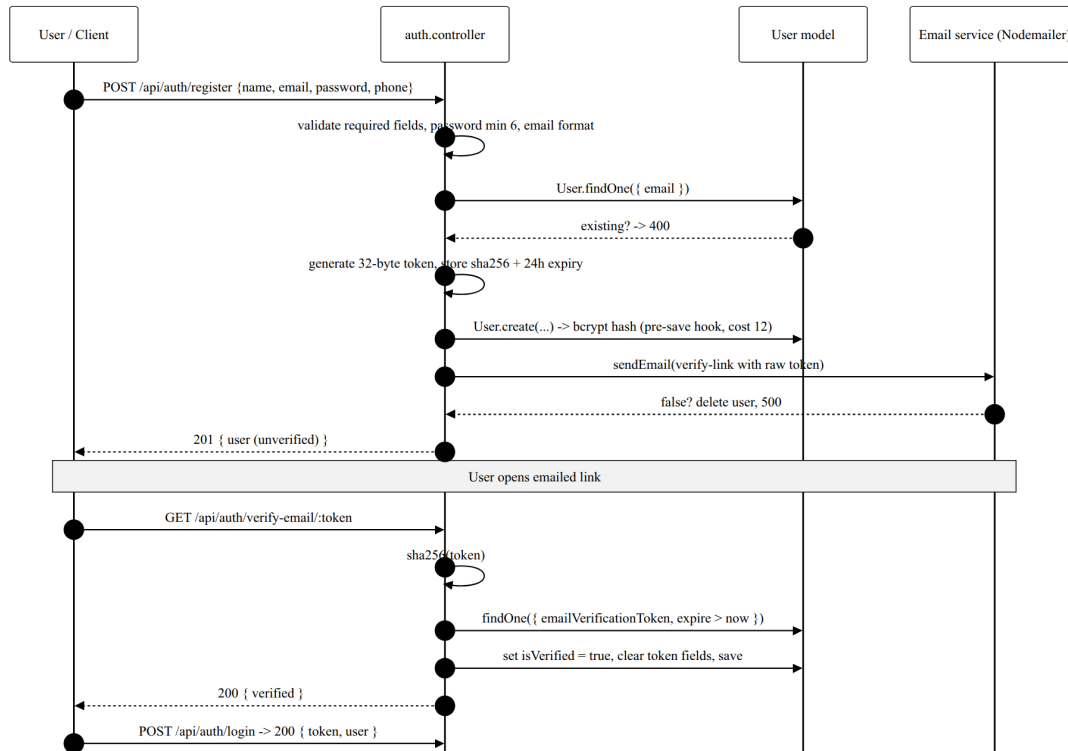


Figure 7.1. Registration and e-mail verification flow.

When `POST /api/auth/register` is received, the controller validates the fields, normalizes the e-mail, checks whether it already exists, generates a 32-byte verification token, stores only its SHA-256 hash with a 24-hour expiry, hashes the password through the pre-save hook, and creates the user with `isVerified: false`. An HTML e-mail containing a verification link is sent through Nodemailer [10]; if the e-mail cannot be sent the user document is deleted so no unverifiable account remains. Clicking the link calls `GET /api/auth/verify-email/:token`, which hashes the presented token and marks the user verified. A resend endpoint generates a fresh token for unverified accounts. Login is refused for unverified accounts (HTTP 403, `isVerified: false` in the response).

7.1.2 Login and Token Issuance

`POST /api/auth/login` selects the user including the (normally excluded) password, compares the presented password with the stored bcrypt [11] hash, refuses blocked and unverified accounts, and on success signs a JWT [12] containing the user identifier with the configured secret and a 30-day expiry (`JWT_EXPIRE`). The client stores the token and sends it as a bearer header on protected calls. The `/api/auth/me` endpoint returns the current user with pets and favourites populated.

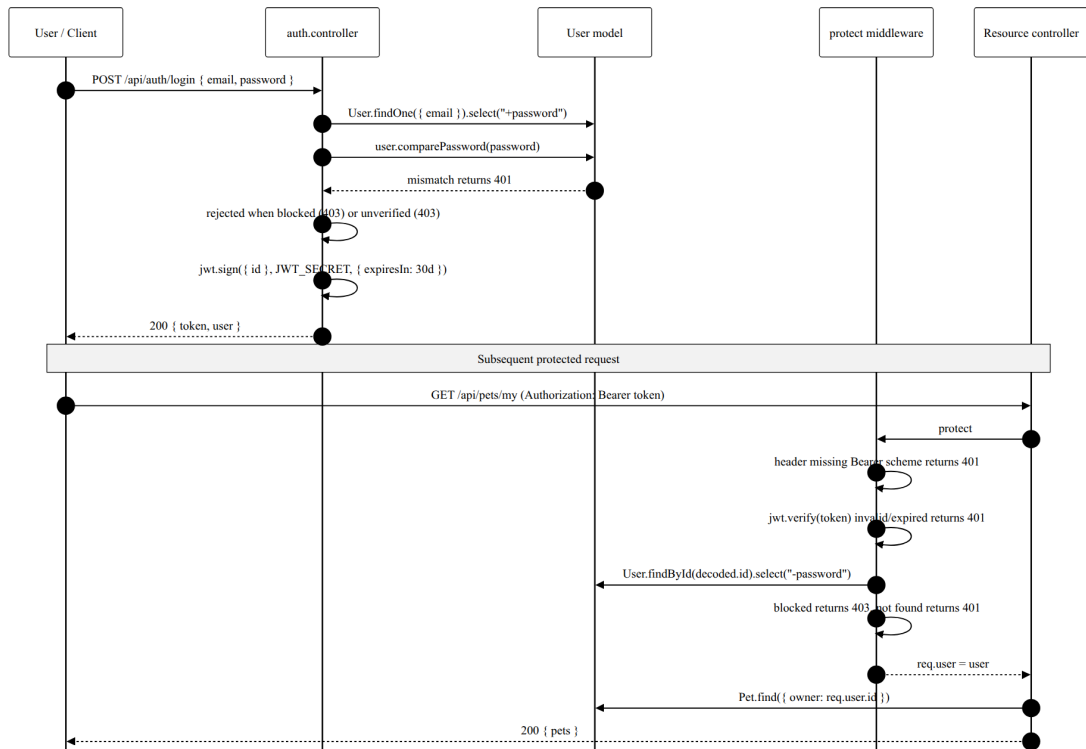


Figure 7.2. Login and JWT issuance flow.

7.1.3 Token Verification and Role-Based Access

Every protected route is wrapped by the `protect` middleware (`backend/middleware/auth.js`). Figure 7.3 shows its decision tree.

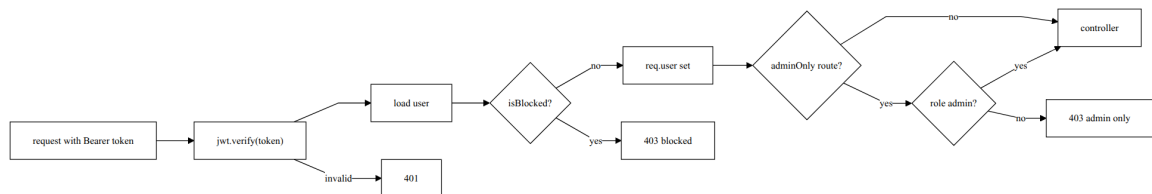


Figure 7.3. Access-control decision tree (protect and adminOnly).

The middleware rejects requests without a bearer header (401), without a configured `JWT_SECRET` (500), with an invalid token (401), with a missing user (401), or with a blocked account (403). On success it assigns `req.user` from the database (excluding the password) and forwards the request. The `adminOnly` middleware then enforces `req.user.role === "admin"` for administrative routes, returning 403 otherwise. Ownership is applied separately inside the controllers: resources are queried with the owner predicate and therefore return 404 for other users' resources.

7.1.4 Password Recovery

`POST /api/auth/forgot-password` always returns a generic “if the e-mail is registered” message so that the endpoint cannot be used to enumerate accounts. A registered e-mail receives a link containing a 32-byte token; the server stores only the SHA-256 hash with a 30-minute expiry. `POST /api/auth/reset-password/:token` hashes the presented token, verifies it and the expiry, stores the new password (bcrypt-hashed by the

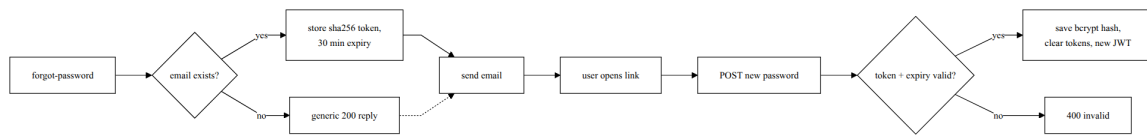


Figure 7.4. Password reset flow.

pre-save hook), clears the token fields, and returns a fresh JWT so the user is logged in immediately. If the e-mail cannot be delivered the stored token is cleared so a stale, unusable token is not left on the account. A separate **change-password** endpoint requires the current password.

7.2 Security Controls

The security controls below are implemented and were verified in the Step 10 test campaign (Chapter 9.4).

Table 7.1. Security controls.

Control	Implementation
Password hashing	bcrypt cost 12 (\$2a\$12\$ hashes verified directly in the database).
Token hygiene	Verification and reset tokens are stored hashed (SHA-256); plain-text tokens appear only in e-mail links.
JWT	Signed bearer tokens with <code>JWT_SECRET</code> ; invalid or expired tokens yield 401.
E-mail verification gate	Unverified accounts cannot log in; forgot-password does not reveal whether an e-mail exists.
Ownership isolation	All user-scoped queries bind to <code>req.user._id</code> ; cross-user access to appointments, reminders, notifications, health records, and vaccinations returns 404.
Role enforcement	<code>adminOnly</code> on all administrative routes; a regular user receives 403.
Input validation	Backend-side checks for required fields, e-mail format, password length, valid ObjectIds, real dates, ownership, and state transitions returning 400/404/500.
CORS	Origin guard accepting the configured client, localhost, and private LAN origins; preflight OPTIONS answered 204.
Security headers	Helmet [13] headers with <code>crossOriginResourcePolicy: cross-origin</code> so uploaded images are embeddable by the frontend origin.
Secret management	Keys live only in <code>backend/.env</code> (git-ignored); only placeholder <code>.env.example</code> is committed; no secrets in repository history.
Blocked accounts	<code>protect</code> and <code>login</code> reject blocked users with 403.

7.3 Security Boundary Verification

The boundary behaviour was tested with two identities: a regular user and an administrator. A regular user calling nine administrative endpoints (`/api/admin/dashboard`, `/api/admin/users`, `/api/admin/pets`, adoption listing and status update, `DELETE /api/admin/users/:id`, breed and veterinarian creation, and lost-and-found moderation) received 403 on every call. A user attempting to read, update, or cancel another user's appointment received 404 because the query is owner-scoped. These results are recorded in the testing chapter (Chapter 9.4).

CHAPTER 8

CORE FEATURE IMPLEMENTATION

This chapter explains the core features of FamiPet using their workflows, data, validation, and security behaviour, each supported by a diagram.

8.1 Digital Pet Identity and QR Code

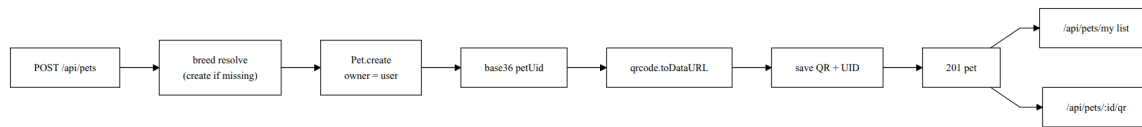


Figure 8.1. Creation of a pet with automatic QR identity.

When a user creates a pet (`POST /api/pets`), the controller accepts either a breed `ObjectId` or a breed name; a name is resolved to a breed document and created on demand if absent. The pet is stored with `owner: req.user.id`. A unique digital pet identifier is then generated as `Date.now().toString(36) - petId.slice(-8) - random` and the `qrcode` [14] package encodes a JSON payload with the pet identifier, unique ID, name, species, and breed into a PNG data URL that is persisted on the pet document. The pet appears in the owner's list (`GET /api/pets/my`), and the dedicated endpoint `GET /api/pets/:id/qr` regenerates a QR code whose payload additionally includes the owner's public name and phone. The digital pet identity is deliberately public-safe: the QR payload contains only identity and public contact fields, never tokens, passwords, or the account reference directly.

8.2 Adoption Management

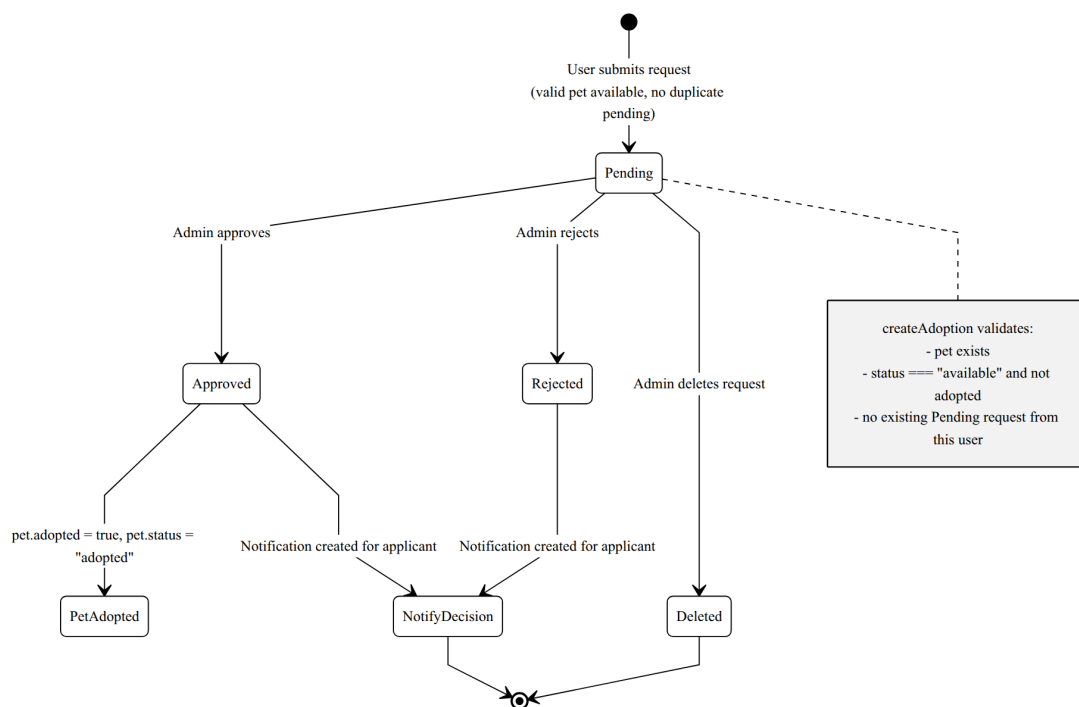


Figure 8.2. Adoption request state diagram.

An adoption request (`POST /api/adoptions`) requires the pet reference, full name, phone, address, and a reason. The controller verifies that the pet exists, is `status: "available"` and not `adopted`, and that the user has no already-pending request for the same pet; a duplicate is rejected. The request is stored in the `Pending` state. Administrators transition the state to `Approved` or `Rejected` through `PUT /api/adoptions/`

:id (admin-only). Approval automatically updates the pet document (adopted: true, status: "adopted"), and every status change creates a notification for the applicant with the pet's name and the new state.

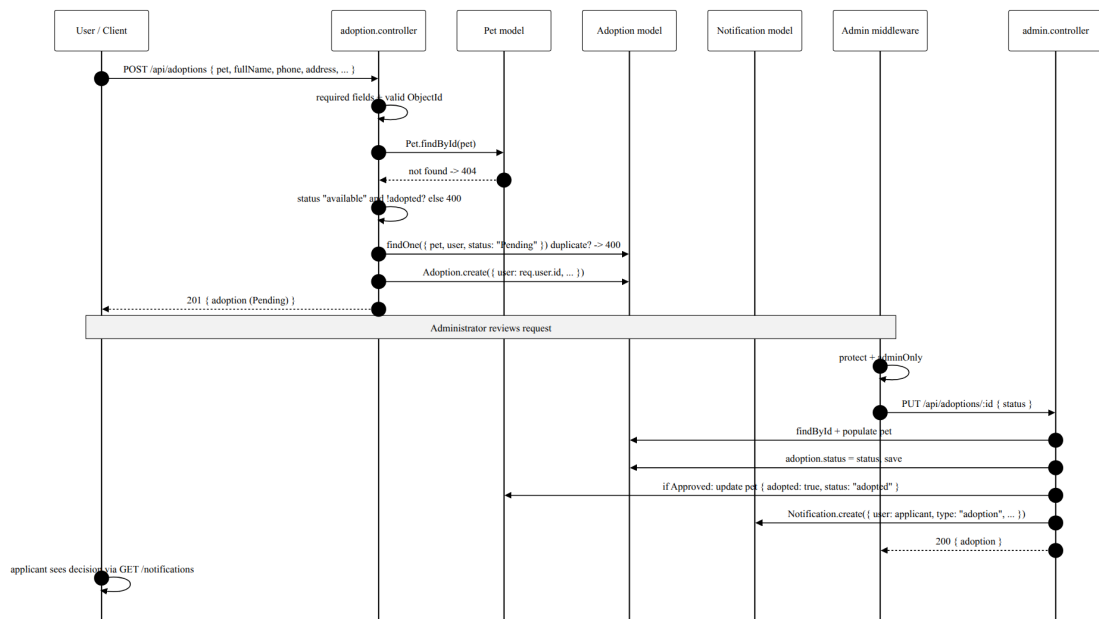


Figure 8.3. Adoption application and approval sequence.

The sequence diagram shows the full path: the user submits the application, the owners own pet records feed the adoption listing, the administrator reviews and approves the request, the backend updates the pet and creates the applicant notification, and the applicant reads both the updated request state and the notification. Notification and pet-record data are stored in MongoDB, so both survive a page reload.

8.3 Appointments, Notifications, and Reminders

Booking an appointment (POST /api/appointments) validates the pet and veterinarian references, requires the pet to belong to the requesting user and the veterinarian to be active, and rejects a double booking: an existing appointment with the same veterinarian, date, and time in the pending or confirmed state returns HTTP 400.

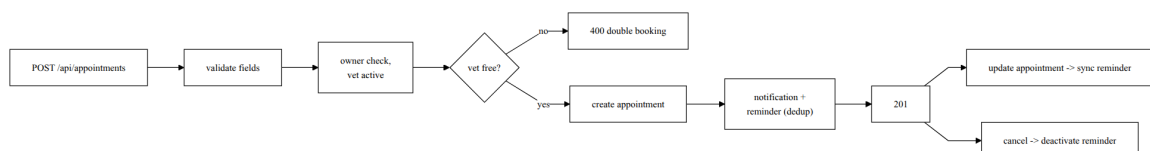


Figure 8.4. Appointment booking, rescheduling, and cancellation flow.

On success the controller creates the appointment, immediately creates a **Notification** (type **appointment**), and creates a **Reminder** (type **appointment**, frequency **once**), but only if an identical active reminder does not already exist, so repeat bookings do not duplicate reminders. Rescheduling an appointment synchronizes the date and time of the matching reminder; cancelling sets the appointment to **cancelled** and deactivates the matching reminder so no stale active reminder remains.

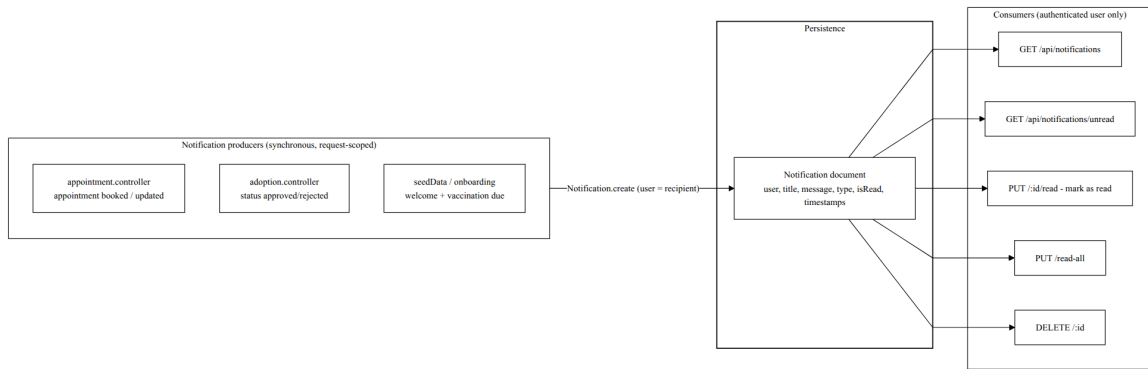


Figure 8.5. Notification creation and consumption flow.

Notifications are the shared feedback channel of the platform. They are created by the adoption and appointment controllers and read through the notification API, which is scoped to the owner and supports an unread filter, mark-as-read, mark-all-as-read, and deletion. The read state persists, so notifications remain correctly unread/read across reloads.

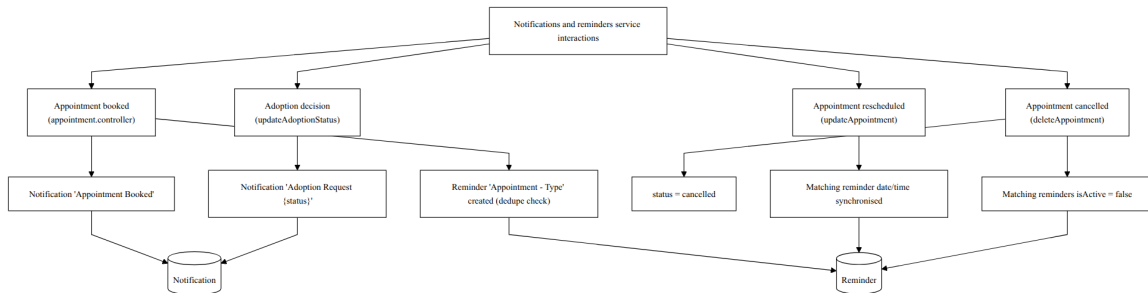


Figure 8.6. Synchronization between appointments, notifications, and reminders.

The synchronization diagram is a summary of the invariant demonstrated by the implementation: booking creates the notification and reminder; rescheduling moves the reminder; cancellation flips the appointment state and deactivates the reminder. No background job is involved; all effects are applied synchronously in the request handler against the same MongoDB store.

8.4 PetGPT – the AI Assistant



Figure 8.7. PetGPT request pipeline with fallback.

The AI subsystem exposes two endpoints. `POST /api/ai/ask` takes a question, loads up to five of the user's own pets (name, species, breed) as context, and calls the Google Gemini [15] REST endpoint (`gemini-1.5-flash`) with a PetGPT system instruction and a 25-second timeout through an `AbortController`. If the API key is missing, the call fails, or times out, a built-in keyword fallback returns safe generic guidance for common topics (dogs, cats, vaccination, diet, exercise). The second endpoint, `POST`

`/api/ai/advice`, is fully rule-based: it requires a pet owned by the user and produces advice from the pet's actual `vaccinated` flag, age, and weight, recommending professional veterinary care when a vaccination or weight record is missing. The assistant therefore never fabricates a diagnosis; its output is either model-generated general advice or profile-derived advice from real records.

8.5 Community and Lost-and-Found

`POST/api/community` creates a post (optionally with an image) bound to the authenticated user. Likes and comments are embedded arrays; like toggling adds or removes the user reference, so re-pressing a like cancels it. Posts are listed publicly, but only when `isActive: true`. Post, comment, and lost-and-found report deletion allow either the owner or an administrator. Lost-and-found reports carry type (`lost/found`), pet attributes, location, date, and contact details, with an `active/resolved` status that administrators can update. Ownership checks are identical in shape to the other subsystems: the report is queried with the owner predicate and the controller verifies `report.user` against `req.user.id` before modification.

8.6 Feature Verification Summary

Each feature described in this chapter was exercised end-to-end in the test campaign of Chapter 9.4: pets and QR, favourites, adoption through to admin approval, appointments with automatic notification and reminder creation, cancellation deactivating the reminder, community posts with images, likes and comments, lost-and-found reporting, health records, vaccinations, and notifications. The persistence, read-back, and cleanup behaviour of every feature was verified against MongoDB.

CHAPTER 9

IMPLEMENTATION

This chapter describes how FamiPet is implemented in code: the repository layout, the backend structure, the data persistence patterns, and the frontend structure.

9.1 Repository Layout

The repository follows a conventional monorepo layout for an Express backend and a browser frontend:

Directory	Content
backend/	Express API; subdirectories <code>config</code> , <code>controllers</code> , <code>middleware</code> , <code>models</code> , <code>routes</code> , <code>utils</code> , <code>uploads</code> , and the entry point <code>server.js</code> .
frontend-react/	The React [5] + TypeScript + Vite [16] + Tailwind [17] client, the project frontend; roadmap phases 1–22 delivered.
docs/	ROADMAP.md (phase plan and status), <code>design.md</code> ; this black book lives under <code>docs/blackbook/</code> .
AGENTS.md, BASELINE.md, Step10-Report.md, test-email.js	Project rules, baseline, and verification evidence.

9.2 Backend Implementation

The backend runs from a single entry point. `server.js` applies Helmet, compression, the CORS origin guard, JSON and URL-encoded body parsing (50 MB limit), and development logging; connects to MongoDB; registers a health route, sixteen route modules under `/api`, static `/uploads`; serves the client build statically on the client port; and ends with a JSON error handler and a 404 handler.

- **Routes** (`routes/`) declare the HTTP surface and bind middleware (`protect`, `admin Only`, `upload`) to each handler. Public read-only endpoints (pets, breeds, veterinarians, community posts, lost-and-found) carry no protection.
- **Controllers** (`controllers/`) implement validation, ownership checks, business effects, and responses. The authentication controller is the largest (roughly 1 280 lines); the admin controller aggregates statistics and moderation operations.
- **Middleware** (`middleware/`) provides JWT authentication and role checks (`auth.js`), the error handler (`errorHandler.js`) that maps `CastErrors` to 404, duplicate-key errors to 400, and Mongoose validation errors to 400, and the Multer upload filter.
- **Models** (`models/`) declare the thirteen schemas with required fields, enums, default values, the unique indexes, and the `bcrypt` pre-save hook.
- **Config** (`config/`) centralizes the database connection, the Nodemailer transporter, the Gemini client, and Cloudinary.
- **Utilities** (`utils/`) provide the database seed data (breeds, an administrator account, and a demo user).

9.3 Persistence Pattern

Every state-changing flow follows the same pattern, which guarantees that what the UI shows after a reload is what the backend wrote:

1. the client sends a request with the bearer token;

2. the controller validates input and ownership against the database;
3. documents are created or updated through the Mongoose models;
4. derived effects (notifications, reminders, pet status transitions) are applied as further document writes;
5. the response returns the persisted data, and the client re-fetches the list from the API whenever it must reflect the new state.

This pattern was verified for every feature in the persistence test campaign (Chapter 9.4).

9.4 Frontend Structure

The React client implements the user interface in phases driven by the roadmap. Phases 1–22 (scaffold, Tailwind, assets, routing, global styles, shared layouts, authentication, dashboard, pet management, health, vaccinations, appointments, veterinarians, reminders, community, lost-and-found, favorites, adoption, breeds, settings, admin panel, PetGPT frontend) are implemented and committed. Notifications are delivered inside phases 10–14 through shared notification components. Phases 23–29 (API integration layer, authentication/state consolidation, UI/UX completion, visual regression, functional regression, Docker/Nginx integration, client consolidation) are planned and not yet implemented.

The React client consumes the same `/api` endpoints described in this black book, so the backend API remains the single source of truth.

CHAPTER 10

TESTING AND VALIDATION

This chapter records the testing performed on FamiPet and the evidence available in the repository. Tests are reported as they were executed; no unverified result is claimed.

10.1 Testing Approach

The project does not ship an in-repository automated test suite. Runtime verification was performed through dedicated browser and persistence test scripts against a live backend and database, and the complete evidence was written to `Step10-Report.md` (dated 09-09-2026), following the structured, evidence-oriented reporting convention of [18]. That campaign ran against the backend at `localhost:5000` and the frontend at `localhost:5502`. The campaign covered four areas: a headless-Chrome sweep of all screens, an end-to-end persistence verification of every feature, an administrator test, and a security boundary test.

10.2 Browser Sweep

All 31 interface screens were scanned in a headless Chrome session at the exact frontend origin: 6 public screens, 13 user screens, and 6 admin screens, plus repeated views for breed details and pet health. The sweep reported zero console errors, zero page errors, and zero failed API calls across every page.

10.3 Database Persistence Verification

The table below records the persistence evidence per feature. Each row reflects an end-to-end check: the API call created the record, the record was found in the database, the read-back through the API rendered in the UI, and where applicable deletion or cleanup was verified.

Table 10.1. Database persistence evidence.

Feature	Result	Verified evidence
Register + e-mail verify	PASS	201; SHA-256 token and expiry stored; login works after verification.
Login / profile update	PASS	Name and city persisted; <code>/auth/me</code> reflects them.
Change / reset password	PASS	New bcrypt <code>\$2a\$12\$</code> hash stored; login with the new password succeeds.
Profile picture	PASS	Avatar URL persisted on the user document and the image renders in the browser.
Pets CRUD + QR	PASS	Document and <code>qrCode</code> persisted; shown in <code>/pets/my</code> as a PNG data URL; delete works.
Favourites	PASS	Persisted in the user favourites collection; toggling off returns the count to zero.
Adoption request	PASS	Created in <code>Pending</code> state; listed in <code>/adoptions/my</code> ; admin approval works.
Appointment	PASS	Document persisted; booking automatically created a notification and a reminder; cancellation deactivated the reminder.
Reminders	PASS	Created, listed, and completion persisted.

Community (image)	post	PASS	Post and image URL persisted; like increments, comment added, public read works; delete works.
Lost and found (image)		PASS	Created active ; document and image persisted; listing and status resolution work.
Health records		PASS	Created, read, updated, deleted.
Vaccinations		PASS	Created, read, upcoming list, updated, deleted.
Notifications		PASS	Appointment auto-notification persisted; read / read-all persisted.
Breeds		PASS	12 breeds with full data displayed as cards and details.

10.4 Administrator Tests

The administrator campaign executed against the admin panel and confirmed: dashboard statistics (9 users, 12 pets, 11 available, 1 adopted, pending adoptions, lost-and-found summaries), the users list and recent users, pets management with owner information, adoption approval (status becomes **Approved**, the pet is automatically marked adopted, and a notification is sent to the applicant), lost-and-found status resolution and restore, and community post deactivation and deletion. All passed.

10.5 Security Tests

Table 10.2. Security test evidence.

Scenario	Result	Observed behaviour
Regular user calls admin routes	PASS	Nine administrative/privileged endpoints each returned 403 for a regular account.
Ownership isolation	PASS	A user could not read, update (404), or cancel (404) another user's appointment; APIs are owner-scoped.
Password hashing	PASS	bcrypt \$2a\$12\$ hashes verified directly in the database for both user and admin accounts.
Invalid token	PASS	Garbage JWT returns 401; missing token returns 401 on protected routes.
CORS	PASS	Only the accept-origin guard permits requests; preflight OPTIONS returns 204 with Authorization and Content-Type allowed.
Secret handling	PASS	No API keys in frontend files; keys live only in the git-ignored backend/.env ; only placeholder .env.example is committed; no secrets in repository history.

10.6 Defects Found and Fixed

The campaign surfaced one real defect: uploaded images were blocked by the browser (`ERR_BLOCKED_BY_RESPONSE.NotSameOrigin`). The cause was Helmet's default `Cross-Origin-Resource-Policy: same-origin`, which prevented the frontend origin from embedding images served by the backend. The fix set `crossOriginResourcePolicy:{policy:"cross-origin"}` in `backend/server.js`. After the fix, avatar and community images rendered (`naturalWidth > 0`) and the full screen sweep was re-run with zero issues. This is the only backend change recorded; the fix is part of the current implementation described in Chapter 4.3.

10.7 Test Artifacts

The automated scripts used for the campaign (`step10-auth.js`, `step10-data.js`, `step10-admin.js`, `step10-pages.js`) reported zero failures across the auth flow, database persistence, admin and security suites, and the 31-page browser sweep. The script files lived in a temporary working directory used for the evaluation and are not part of the repository; the results are recorded in `Step10-Report.md` in the repository root.

CHAPTER 11

DEVELOPMENT PROCESS

This chapter documents how FamiPet was developed: the workflow rules, the phase-based roadmap, and the project timeline reconstructed from the repository history.

11.1 Development Workflow

The project follows the workflow encoded in `AGENTS.md`: inspect before changing, treat the backend as the source of truth for identity and ownership, preserve existing functionality, use real data only, persist through the database, enforce security on the backend, make the smallest correct change, test the complete flow, and never claim unverified results. Each phase of work is completed as an atomic unit: inspect, implement, test, fix failures, update documentation, commit, tag and push when specified, verify a clean working tree, and stop.

Every change must reuse existing models, routes, controllers, and utilities instead of duplicating endpoints or bypassing authentication. The role and ownership of every resource are enforced on the backend, never by hiding UI controls. The disciplined, phase-based engineering practices follow the software-engineering fundamentals collected by [19].

11.2 Phase-Based Roadmap

The implementation is tracked in `ROADMAP.md` as numbered phases. The frontend phase status below is taken directly from the roadmap. Each implemented phase is delivered as an atomic commit; the *Commit* column records the feature commit that completes the phase.

Table 11.1. Roadmap phase status.

Phase	Name	Status	Commit
1	Scaffold (Vite + React + TS)	Complete	80cc21d
2	Tailwind setup	Complete	80cc21d
3	Assets and theme	Complete	80cc21d
4	Routing	Complete	c5f23f3
5	Global styles and shared layout	Complete	be87712
6	Shared layout components	Complete	1a2cf5e
7	Authentication	Complete	9b44727
8	Dashboard	Complete	d2f6952
9	Pet management	Complete	38c6aa9
10	Health	Complete	17fc630
11	Vaccinations	Complete	3d10701
12	Appointments	Complete	e003c98
13	Veterinarians	Complete	721f03f
14	Reminders	Complete	56a1c29
15	Community	Complete	5dd0fef
16	Lost & found	Complete	d637cd9
17	Favourites	Complete	d7a6f10
18	Adoption	Complete	70293d0
19	Pet breeds	Complete	526f78e
20	Settings	Complete	ed4d6cb
21	Admin panel	Complete	d18c336
22	AI / PetGPT frontend	Complete	d26e955
23	API integration layer	Planned	—
24	Auth / state consolidation	Planned	—
25	UI/UX completion	Planned	—
26	Visual regression	Planned	—

27	Functional regression	Planned	—
28	Docker / Nginx integration	Planned	—
29	Client consolidation	Planned	—

11.3 Timeline and Project Gantt

The project was initiated in July 2026. The initial period (July and August) covered planning, requirements analysis, and design, and is recorded in the project documentation; the repository itself does not carry those commits. The version-controlled implementation begins on 2026-09-07 with the backend commits. The Step-10 verification campaign was executed on 2026-09-09, and the React frontend phases were implemented from 2026-09-21 through 2026-09-22, with phases 23–29 still planned. The roadmap does not record dates, so the timeline below is reconstructed from the repository history and the documented project start, following the scheduling conventions of the PMBOK [20]; the Gantt chart itself was rendered with Mermaid [21].

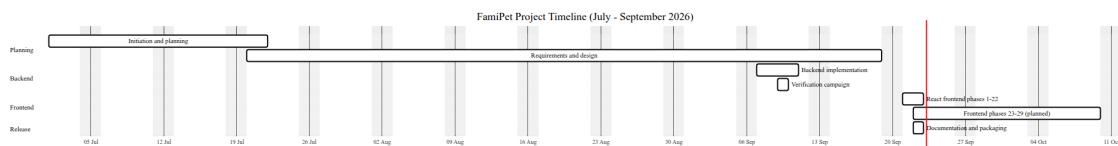


Figure 11.1. Project timeline and Gantt chart (July–September 2026).

The key milestones are summarised in Table 11.2.

Table 11.2. Key project milestones.

Activity	Period	Evidence
Initiation and planning	Jul 2026	Project brief and design direction.
Requirements and design	Jul–Aug 2026	Requirements, architecture, and design documentation.
Backend implementation	07–10 Sep 2026	Initial commits; the sixteen route modules covering all documented features.
Verification campaign	09 Sep 2026	<code>Step10-Report.md</code> : browser sweep, persistence, admin, and security tests.
React frontend phases 1–22	21–22 Sep 2026	Roadmap phase table (Table 11.1) with commit hashes.
Documentation and packaging	22 Sep 2026	Black book, project skills, and graphify artifacts committed.

CHAPTER 12

CONCLUSION AND FUTURE SCOPE

This chapter summarises what was delivered, what the verification showed, and the planned future work.

12.1 Summary of the Delivered System

FamiPet delivers a complete, authenticated pet-care and adoption platform. The backend, built with Node.js, Express, MongoDB, and Mongoose, exposes sixteen route modules that cover account management with e-mail verification and secure password recovery; pet management with a unique digital pet identity and QR code; health records and vaccinations; veterinary appointment booking with double-booking protection and automatic notification and reminder creation; care reminders; favourites; community posts with likes and comments; lost-and-found reports; adoption applications with administrator approval; a per-user notification hub; and an AI pet-care assistant grounded in the user's real pet records.

The system enforces the rules of the project brief on the backend: user identity comes from the signed token, every user-scoped query binds to the authenticated user, administrative actions are gated by role, data is persisted in MongoDB and read back through the API, and validation errors are returned as explicit HTTP responses. The Step 10 verification campaign confirmed persistence, ownership isolation, role-based 403s, bcrypt password hashing, JWT handling, CORS behaviour, and a clean browser sweep over all 31 screens.

12.2 Observations

Three implementation choices deserve emphasis because they affect future work. First, the AI assistant is the only feature that depends on an external paid service; it falls back to deterministic guidance when the service is unavailable, so the platform never breaks. Second, notifications and reminders are produced synchronously by the controllers, which keeps the system simple and fully consistent at request time, at the cost of no background scheduling or real-time push. Third, the React client consumes the backend API as the single source of truth, so the pages always present exactly the persisted state the backend returns.

12.3 Future Scope

The planned future work is recorded in the roadmap. The items below are planned, not implemented.

Table 12.1. Planned future work.

Area	Planned work
API integration layer	Phase 23: formal typed API client for the React app.
Auth and state management	Phase 24: consolidation of authentication and state management across migrated pages.
UI/UX completion	Phase 25: finish the remaining interface surfaces.
Visual regression	Phase 26: pixel-level regression for the frontend UI.
Functional regression	Phase 27: end-to-end regression across all features.
Docker / Nginx	Phase 28: containerization and reverse-proxy deployment.
Client consolidation	Phase 29: decommission the legacy browser client once regression phases are green.

Background scheduling	Considered: convert reminders into scheduled jobs and add push notifications using the unused <code>node-cron</code> and <code>socket.io</code> dependencies.
Provider expansion	Considered: implement the OpenAI adapter described in <code>.env.example</code> as an alternative AI provider behind the Google default.

12.4 Closing Statement

The black book documents the FamiPet project as it exists in the repository: its requirements, architecture, data model, security, features, implementation, testing evidence, and development process. Implemented, planned, and considered items have been clearly distinguished throughout. The system is feature-complete on the backend, and the React frontend implements the delivered roadmap phases; the backend remains the source of truth for all data and authorization.

REFERENCES

The sources cited in this black book. All references relate to verified technologies, documentation and literature used in the project.

- [1] Node.js Node – OpenJS Foundation, “Node.js documentation.” <https://nodejs.org/en/docs>. Accessed September 2026.
- [2] Express.js Foundation, “Express – fast, unopinionated, minimalist web framework for node.js.” <http://expressjs.com>. Accessed September 2026.
- [3] E. Yourdon, *Modern Structured Analysis*. Prentice-Hall, 1989. Data-flow diagram conventions.
- [4] C. Gane and T. Sarson, *Structured Systems Analysis: Tools and Techniques*. Prentice-Hall, 1979. Data-flow diagram conventions.
- [5] I. Meta Platforms, “React – a javascript library for building user interfaces.” <https://react.dev/>. Accessed September 2026.
- [6] Expressjs, “Multer – node.js middleware for handling multipart/form-data.” <https://www.npmjs.com/package/multer>. Accessed September 2026.
- [7] Cloudinary, “Cloudinary documentation – image and video platform.” <https://cloudinary.com/documentation>. Accessed September 2026.
- [8] I. MongoDB, “Mongodb manual.” <https://www.mongodb.com/docs/>. Accessed September 2026.
- [9] Mongoose ODM, “Mongoose – elegant mongodb object modeling for node.js.” <https://mongoosejs.com/docs>. Accessed September 2026.
- [10] Nodemailer, “Nodemailer – sending e-mails with node.js.” <https://nodemailer.com/about/>. Accessed September 2026.
- [11] Kelektiv, “bcrypt.js – a library to help you hash passwords.” <https://github.com/kelektiv/node.bcrypt.js>. Accessed September 2026.
- [12] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT) – RFC 7519.” <https://datatracker.ietf.org/doc/html/rfc7519>, 2015. Internet Engineering Task Force.
- [13] Helmet, “Helmet – secure express apps by setting http headers.” <https://helmetjs.github.io/>. Accessed September 2026.
- [14] Node-qrcode, “qrcode – qr code/2d barcode generator for node.js.” <https://www.npmjs.com/package/qrcode>. Accessed September 2026.
- [15] Google, “Google generative ai – gemini api.” <https://ai.google.dev/>. Accessed September 2026.
- [16] Vite, “Vite – next generation frontend tooling.” <https://vite.dev/guide/>. Accessed September 2026.
- [17] T. CSS, “Tailwind css – rapidly build modern websites without ever leaving your html.” <https://tailwindcss.com/docs>. Accessed September 2026.
- [18] IEEE, “Ieee reference guide.” <https://ieeauthorcenter.ieee.org/>. Accessed September 2026.

- [19] I. Sommerville, *Software Engineering*. Pearson, 10 ed., 2016.
- [20] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. Project Management Institute, 7 ed., 2021. Project scheduling and Gantt chart conventions.
- [21] Mermaid, “Mermaid – javascript-based diagramming and charting.” <https://mermaid.js.org/>. Accessed September 2026; Gantt and flowchart diagrams rendered with Mermaid CLI.

APPENDIX A – REST API SUMMARY

A catalogue of the HTTP surface exposed by the FamiPet backend, as implemented in the route modules.

Authentication and Users

Endpoint	Method	Access
/api/auth/register	POST	Public
/api/auth/login	POST	Public
/api/auth/verify-email/:token	GET	Public
/api/auth/resend-verification	POST	Public
/api/auth/forgot-password	POST	Public
/api/auth/reset-password/:token	POST	Public
/api/auth/me	GET	Protect
/api/auth/profile	PUT	Protect
/api/auth/change-password	PUT	Protect
/api/users	GET	Protect
/api/users/:id	GET	Public (profile)
/api/users/favorites/:petId	POST	Protect
/api/users/avatar	POST	Protect (upload)

Pets, Breeds, and Favourites

Endpoint	Method	Access
/api/pets	GET	Public
/api/pets/featured	GET	Public
/api/pets/my	GET	Protect
/api/pets	POST	Protect
/api/pets/:id	GET	Public
/api/pets/:id	PUT DELETE	/ Protect (owner)
/api/pets/:id/qr	GET	Protect (owner)
/api/breeds	GET	Public
/api/breeds/:id	GET	Public
/api/breeds	POST	Protect + adminOnly
/api/breeds/:id	PUT DELETE	/ Protect + adminOnly
/api/favorites	GET / POST	Protect
/api/favorites/:id	DELETE	Protect

Care: Health, Vaccinations, Veterinarians, Appointments, Reminders

Endpoint	Method	Access
/api/health	GET / POST	Protect (owner)
/api/health/:id	GET / PUT / DELETE	Protect (owner)
/api/vaccinations	GET / POST	Protect (owner)
/api/vaccinations/upcoming	GET	Protect (owner)
/api/vaccinations/:id	PUT / DELETE	Protect (owner)
/api/veterinarians	GET	Public
/api/veterinarians/:id	GET	Public
/api/veterinarians	POST	Protect + adminOnly
/api/veterinarians/:id	PUT / DELETE	Protect + adminOnly
/api/appointments	GET / POST	Protect (owner)
/api/appointments/:id	PUT / DELETE	Protect (owner)
/api/reminders	GET / POST	Protect (owner)
/api/reminders/:id	PUT / DELETE	Protect (owner)
/api/reminders/:id/complete	PUT	Protect (owner)

Community, Lost-and-Found, Adoption, Notifications, AI

Endpoint	Method	Access
/api/community	GET	Public (active posts)
/api/community/:id	GET	Public
/api/community	POST	Protect (upload)
/api/community/:id	PUT / DELETE	Protect (owner)
/api/community/:id/like	POST	Protect
/api/community/:id/comments	POST	Protect
/api/community/:id/comments/:commentId	DELETE	Protect (owner)
/api/lost-found	GET	Public
/api/lost-found/:id	GET	Public
/api/lost-found	POST	Protect (upload)

/api/lost-found/:id	PUT	/ Protect (owner or admin)
	DELETE	
/api/adoptions	GET	Protect + adminOnly
/api/adoptions/my	GET	Protect (owner)
/api/adoptions	POST	Protect
/api/adoptions/:id	PUT	/ Protect + adminOnly
	DELETE	
/api/notifications	GET	Protect (owner)
/api/notifications/unread	GET	Protect (owner)
/api/notifications/read-all	PUT	Protect (owner)
/api/notifications/:id/read	PUT	Protect (owner)
/api/notifications/:id	DELETE	Protect (owner)
/api/ai/ask	POST	Protect
/api/ai/advice	POST	Protect

Admin

Endpoint	Method	Access
/api/admin/dashboard	GET	Protect + adminOnly
/api/admin/users	GET	Protect + adminOnly
/api/admin/users/recent	GET	Protect + adminOnly
/api/admin/users/:id/block	PUT	Protect + adminOnly
/api/admin/users/:id	DELETE	Protect + adminOnly
/api/admin/pets	GET	Protect + adminOnly
/api/admin/pets/:id	DELETE	Protect + adminOnly
/api/admin/lost-found	GET	Protect + adminOnly
/api/admin/lost-found/:id/status	PUT	Protect + adminOnly
/api/admin/lost-found/:id	DELETE	Protect + adminOnly
/api/admin/community	GET	Protect + adminOnly
/api/admin/community/:id/status	PUT	Protect + adminOnly
/api/admin/community/:id	DELETE	Protect + adminOnly

System

Endpoint	Method	Access
/api/status	GET	Public (health check)
/	GET	Public (service banner)
/uploads/*	GET	Public (static images)

APPENDIX B – CONFIGURATION

The runtime configuration of the FamiPet backend, as declared in `backend/.env.example` and read by the code.

Environment Variables

Variable	Default	Purpose
PORT	5000	HTTP port for the API process.
MONGODB_URI	mongodb://localhost:27017/animal_planet	MongoDB connection string.
JWT_SECRET	(required)	Secret used to sign and verify bearer tokens.
JWT_EXPIRE	30d	Token expiry duration.
CLIENT_URL	http://localhost:5500	Frontend origin(s), comma-separated, used by CORS and e-mail links.
GEMINI_API_KEY	(optional)	Google Gemini key; the PetGPT pipeline falls back to built-in answers when absent.
EMAIL_SERVICE	gmail	Nodemailer service name.
EMAIL_USER	(SMTP)	Sender account for Nodemailer.
EMAIL_PASS	(SMTP)	App password for the e-mail account.
CLOUDINARY_CLOUD_NAME	(optional)	Cloudinary account for image hosting.
CLOUDINARY_API_KEY	(optional)	Cloudinary API key.
CLOUDINARY_API_SECRET	(optional)	Cloudinary API secret.
PETGPT_PROVIDER	(commented)	Declared OpenAI option; the active implementation uses the Google provider only.

Startup and Defaults

The backend binds the API to **PORT** (default 5000). It connects to MongoDB through **MONGODB_URI**; if the variable is missing the connection falls back to the local default in `server.js`. The server also listens on the client port to serve the frontend directory as a fallback when the Live Server/Vite dev server is not running; if that port is already in use the fallback is skipped silently. The **uploads** directory is served statically at **/uploads**, and **CLIENT_URL** (or an allowed origin) is used to build the verify-e-mail and reset-password links sent by e-mail.

Seed Data

The `seedData.js` utility populates the database with the breed catalogue, an administrator account, a demo user account, two veterinarians with clinics and availability, and a set of sample appointments, adoption requests, and notifications so the application can be evaluated immediately after setup.